

Washify

Technical Documentation

Multi-tenant laundry SaaS · React + Supabase + n8n + AI

Prepared as an interview reference for an AI / Automation Specialist role

Table of Contents

1. Project Overview
2. System Architecture
3. Tools Used (and why)
4. n8n Concepts — Webhook vs HTTP Request, GET vs POST
5. The n8n Workflows, Node by Node
 - a. Laundry-Done SMS Notifier
 - b. FAQ Chatbot (RAG)
 - c. FAQ Ingest
 - d. Messenger Inbound (capture + link PSID)
 - e. Messenger Send
 - f. AI Message Drafting
6. How the AI / RAG Pipeline Works
7. Multi-Tenant Security & Secrets
8. SMS vs Messenger — Channel Constraints
9. The Application (Frontend)
10. Data Model
11. Challenges & How They Were Solved
12. Glossary
13. Interview Talking Points & Likely Questions

1. Project Overview

Washify is a lightweight, multi-tenant SaaS-style laundry manager. Any shop owner signs in with Google, gets their own private workspace, logs orders, notifies customers when their laundry is done (via SMS or Facebook Messenger), tracks sales, and answers customer questions with an AI chatbot. It runs entirely on free tiers.

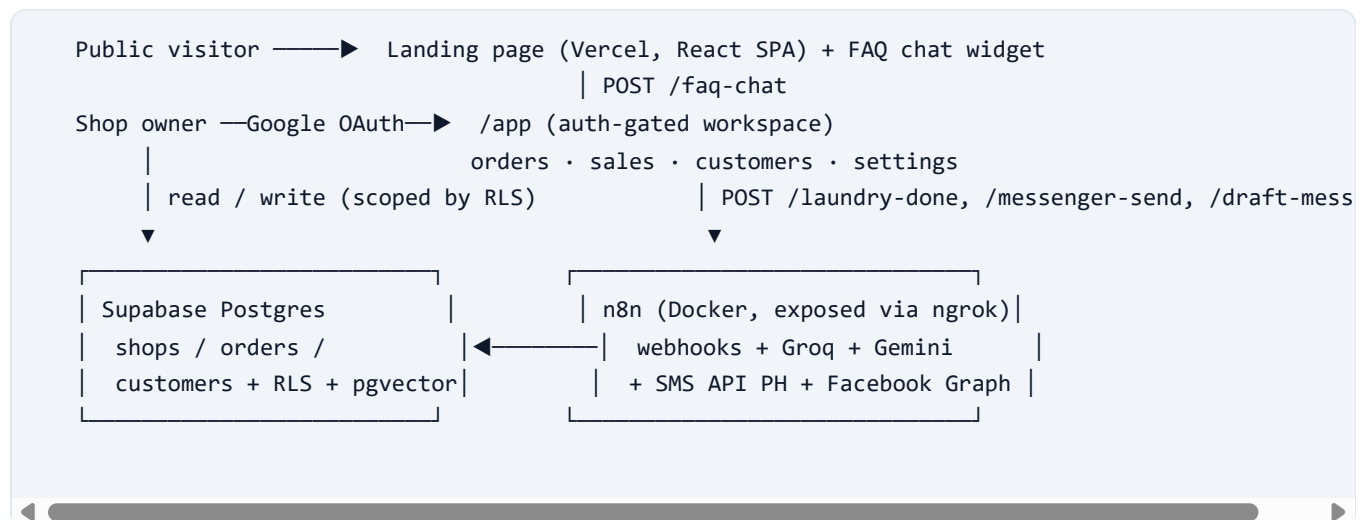
The product is real, but its purpose is to demonstrate three competencies end-to-end:

- **Workflow automation** — event-driven n8n workflows behind webhooks.
- **Applied AI** — a Retrieval-Augmented Generation (RAG) chatbot and an LLM message-drafting feature.
- **Real system design** — a multi-tenant database with security enforced in the database, not the UI.

The problem it models

A small laundromat has three recurring pains: customers asking "is it ready yet?", staff manually texting people, and the same FAQs (hours, price, turnaround) being asked all day. Washify automates all three and keeps every shop's data isolated, so it behaves like a true SaaS product.

2. System Architecture



Why this shape: the React app talks *directly* to Supabase using the signed-in user's token; Row Level Security guarantees isolation, so there is no custom backend to maintain. Anything that is "automation" (sending an SMS, answering with AI, drafting a message) is pushed behind an **n8n webhook**. That separation is the core design decision: the app never holds an API key, and a channel can be swapped by editing one n8n node without touching the app.

3. Tools Used (and why)

Tool	Role in Washify	Why chosen
React + TypeScript (Vite)	Frontend SPA: landing page + authed workspace	Fast, typed, deploys as static files
Tailwind CSS	Styling via semantic design tokens	Consistent, no stray CSS files
React Router	Public <code>/</code> vs protected <code>/app</code> routes	Standard SPA routing + route guard
Supabase	Auth (Google), Postgres DB, Row Level Security, pgvector, Realtime	One free backend for auth + relational data + vector store
n8n	All automations (webhooks, API calls, AI orchestration)	Visual, swappable, the "automation specialist" showcase
SMS API PH	Sends the actual SMS	Free Philippine gateway, no opt-in needed
Facebook Graph API	Sends Messenger messages	Optional second notification channel
Groq (Llama 3.3 70B)	LLM that answers FAQs and drafts messages	Free, very fast inference
Google Gemini (text-embedding-004)	Turns text into 768-dim vectors for search	Free embeddings to pair with Groq
Vercel	Hosts the app, auto-deploys from GitHub	Zero-config static hosting
ngrok	Exposes local n8n over HTTPS so Facebook can reach it	Free static tunnel for webhooks
SheetJS (xlsx)	Client-side Excel export of sales	No server needed; loaded on demand
qrcode.react	Renders the Messenger connect QR on receipts	Offline QR, no external image calls

4. n8n Concepts — Webhook vs HTTP Request, GET vs POST

This is the section most worth understanding cold, because two pairs of ideas are easy to confuse.

Webhook node vs HTTP Request node

	Webhook node	HTTP Request node
Direction	Receives — n8n is the server, waiting to be called	Sends — n8n is the client, calling someone else's API
Analogy	A door someone knocks on	A phone call n8n makes
Position in a workflow	Always the <i>start</i> (a trigger)	<i>A step in the middle</i>
In Washify	App / Meta call into n8n (<code>/laundry-done</code> , <code>/faq-chat</code> , etc.)	n8n calls out to SMS API PH and Facebook

So a single workflow often has both: a **Webhook** receives the request, then an **HTTP Request** calls an external service to do the work.

GET vs POST (the HTTP methods)

	GET	POST
Purpose	Retrieve / read data	Send / create / trigger an action
Where data rides	In the URL query string (<code>?key=value</code>)	In the request body (usually JSON)
Has a body?	No	Yes
Repeat-safe?	Yes (idempotent, read-only)	No (each call may create/change something)

How that combines in Washify

- **Webhook · POST** The normal case — the app sends a JSON body to n8n (e.g. `{ phone, message }`). Used by every app-triggered workflow.
- **Webhook · GET** Used once, for **Facebook's webhook verification**. Meta sends a GET with a `hub.challenge` query value; n8n must echo it back as plain text to prove it owns the URL. No body involved — that's why it's a GET.
- **HTTP Request · POST** n8n sending data out: posting the SMS to SMS API PH, or posting the message to the Facebook Graph Send API.
- **HTTP Request · GET** Would fetch data from an external API (not needed in the current workflows, since every external call *creates* something — a sent message).

Test URL vs Production URL. While editing, n8n gives a `/webhook-test/...` URL that listens for one call after you click *Execute*. Once you **Activate** the workflow, the permanent `/webhook/...` URL works continuously. The app always calls the production URL.

Other n8n building blocks used

- **Edit Fields (Set)** — builds or reshapes data (e.g. normalize a phone number, pull a value out of a nested payload).
- **Respond to Webhook** — sends the HTTP response back to whoever called the webhook (so the app gets `{ status: "sent" }`).
- **Credentials** — n8n's encrypted store for API keys. Nodes reference a credential by name, so exported workflow JSON never contains the real key.
- **AI sub-nodes** — Chat Model, Embeddings, Vector Store and Data Loader attach *underneath* a chain node to give it a brain, a memory, and a retriever.

5. The n8n Workflows, Node by Node

Washify uses six workflows. Each is introduced with what it does, then broken down node by node.

a. Laundry-Done SMS Notifier (washify-sms)

What it does: when staff tap "Text" on a ready order, the app posts the phone + message and this workflow sends a real SMS. Flow: `Webhook` → `Edit Fields` → `HTTP Request` → `Respond`.

Node	Type	What it does
Webhook	Webhook (POST <code>/laundry-done</code>)	Receives <code>{ phone, message }</code> from the app.
Edit Fields	Set	Normalizes the phone to E.164 (<code>0917...</code> → <code>+639...</code>) and carries the message through.
HTTP Request	HTTP Request (POST)	Calls the SMS API PH endpoint with the <code>x-api-key</code> header to actually send the text.
Respond to Webhook	Respond	Returns <code>{ status: "sent" }</code> so the app can show a ✓.

b. FAQ Chatbot — RAG (washify-chatbot)

What it does: answers public questions grounded only in the shop's FAQ. Flow: `Webhook` → `Q&A Chain` → `Respond`, with three AI sub-nodes feeding the chain.

Node	Type	What it does
Webhook	Webhook (POST <code>/faq-chat</code>)	Receives <code>{ question }</code> from the chat widget.
Question and Answer Chain	LangChain Retrieval QA	Orchestrates RAG: embeds the question, retrieves matching FAQ, asks the LLM, returns the answer.
Groq Chat Model	Sub-node (LLM)	The model that writes the answer (Llama 3.3 70B).
Vector Store Retriever	Sub-node (retriever)	Finds the most relevant FAQ chunks for the question.
Supabase Vector Store	Sub-node (vector DB)	Holds the embedded FAQ in pgvector; does the similarity search.
Embeddings Google Gemini	Sub-node (embeddings)	Turns the question into a 768-dim vector to compare against stored chunks.
Respond to Webhook	Respond	Returns <code>{ answer }</code> to the widget.

c. FAQ Ingest (washify-faq)

What it does: a one-time (re-run on change) workflow that loads the FAQ text into the vector store.

Flow: Manual Trigger → Edit Fields → Supabase Vector Store (insert) .

Node	Type	What it does
When clicking Execute	Manual Trigger	You run it by hand whenever the FAQ changes (no webhook).
Edit Fields	Set	Holds the FAQ knowledge-base text.
Supabase Vector Store	Vector store (insert mode)	Stores the embedded chunks into the <code>documents</code> table.
Embeddings Google Gemini	Sub-node	Converts each chunk into a vector.
Default Data Loader	Sub-node	Splits the long text into chunks and tags metadata (shop id).

d. Messenger Inbound — capture & link PSID (n8n-fb-message)

What it does: handles Facebook's verification and incoming messages, and links a customer's Messenger ID (PSID) to an order. It is **two independent chains** on one path.

Node	Type	What it does
Verify	Webhook (GET <code>/messenger</code>)	Receives Meta's verification GET.
Respond to Webhook	Respond (text)	Echoes back <code>hub.challenge</code> to prove URL ownership.
Incoming	Webhook (POST <code>/messenger</code>)	Receives every message the Page gets.
Extract PSID	Set	Pulls out the sender's PSID and normalizes the message text into an order <code>code</code> .
Link to Order	Supabase (update)	Saves the PSID onto the order whose <code>order_code</code> matches the code the customer sent.

Why match on the typed code instead of a hidden referral? Facebook's `m.me?ref=` referral events are unreliable and need extra subscriptions. The message *text* always arrives, so matching the order code the customer sends is far more dependable.

e. Messenger Send (n8n-messenger-notify)

What it does: sends a Messenger message, mirroring the SMS workflow. Flow: Webhook → HTTP Request → Respond .

Node	Type	What it does
Webhook	Webhook (POST <code>/messenger-send</code>)	Receives <code>{ psid, message }</code> from the app.
HTTP Request	HTTP Request (POST)	Calls the Facebook Graph Send API with a Bearer Page token; <code>messaging_type: RESPONSE</code> (within the 24-hour window).
Respond to Webhook	Respond	Returns <code>{ status: "sent" }</code> .

f. AI Message Drafting (washify-draft)

What it does: turns an order into a friendly notification message using the LLM. Flow: `Webhook` → `Basic LLM Chain` → `Respond` .

Node	Type	What it does
Webhook	Webhook (POST <code>/draft-message</code>)	Receives order context (name, quantity, amount, paid, channel, char limit).
Basic LLM Chain	LangChain LLM Chain	Holds the prompt + rules and asks the model for one short message.
Groq Chat Model	Sub-node (LLM)	Generates the message text.
Respond to Webhook	Respond	Returns <code>{ message }</code> (built with <code>JSON.stringify</code> so quotes/newlines are safe).

6. How the AI / RAG Pipeline Works

Retrieval-Augmented Generation means: fetch the relevant facts first, then let the model answer from them — so it stays grounded and doesn't invent policies.

1. **Ingest (once / on change):** split the FAQ into chunks → turn each into a 768-dimension vector with Gemini → store in Supabase `pgvector` .
2. **Query (every question):** embed the question → cosine-similarity search for the closest chunks → hand them to Groq with a strict system prompt → return the answer.

Why two AI providers? Groq is fast and free for *generation*; Gemini's `text-embedding-004` is free for *embeddings* (Groq doesn't offer embeddings). Using the right tool for each half is a deliberate, cost-aware choice.

The **AI message drafting** feature uses the same Groq model but no retrieval — it's a single prompt that turns structured order data into a warm, length-limited message.

7. Multi-Tenant Security & Secrets

Row Level Security (RLS)

Every owner is one shop, keyed so `shops.id = auth.uid()`. Each row in `orders` and `customers` carries a `shop_id`, and RLS policies allow access only where `shop_id = auth.uid()`. Isolation is enforced **in the database**, so even a forged client request cannot read another shop's data. A Postgres trigger auto-creates a shop row on first sign-in.

Secret handling

- API keys live **only in n8n's encrypted credential store** (or, for the frontend, the Supabase *anon* key, which is safe to expose because RLS protects it).
- Workflow JSONs committed to the repo use **placeholders** (`YOUR_API_KEY`), never real keys.
- Lesson learned: a key committed once stays in git history — redacting later isn't enough, you must **rotate** it.

8. SMS vs Messenger — Channel Constraints

	SMS (SMS API PH)	Facebook Messenger
Recipient identified by	Phone number (already collected)	PSID (only obtained after the customer messages the Page)
Can you message cold?	Yes	No — customer must opt in first
Timing limit	None (1 SMS / 10 s rate limit)	24-hour window after the customer's last message
Message length	160 chars (1 segment)	2000 chars
Cost	Free gateway	Free, but policy-restricted

Key insight: SMS needs no opt-in, so it's the primary channel for "your laundry is ready." Messenger is an opt-in extra. Matching the channel to its constraint is the engineering point.

9. The Application (Frontend)

React + TypeScript SPA, organized feature-first (`features/landing` , `orders` , `dashboard` , `customers` , `settings` , `auth` , `faq-chat`). The authed workspace and heavy libraries are **code-split**, so public visitors download almost none of the app code.

Post-login features

- **Orders** — add (with customer auto-fill), edit, delete, status pipeline (Received → Washing → Ready → Picked up), paid toggle, search + filter.
- **Notify** — SMS or Messenger, with an "AI draft" button and a Messenger connect QR.
- **Sales dashboard** — revenue, loads/kg, orders, outstanding balance, average order value, repeat-customer rate, week-over-week change, charts, and Excel export.
- **Customers** — directory auto-built from orders, with visit counts.
- **Printable claim stub** — order code + Messenger QR.
- **Settings** — shop name, per-load / per-kilo pricing, price, Facebook Page ID.
- **Realtime** — orders update live via Supabase Realtime (e.g. a PSID captured by n8n appears without a refresh).

10. Data Model

Table	Key columns	Notes
shops	id (= auth.uid()), shop_name, pricing_mode, price_per_load, price_per_kg, messenger_page_id, onboarded	One row per owner.
orders	id, shop_id, order_code, customer_name, phone, num_loads, amount_due, status, paid, texted_at, messenger_psid	RLS-scoped by shop_id.
customers	id, shop_id, name, phone, visit_count, last_visit, messenger_psid	Auto-upserted on each order.
documents	id, content, metadata, embedding vector(768)	The FAQ vector store (pgvector).

11. Challenges & How They Were Solved

- **Empty webhook response crashed the app.** An SMS call returned `200` with an empty body; `JSON.parse("")` threw. Fix: the n8n client now tolerates empty/non-JSON responses.
- **Leaked API key.** A key was committed in a workflow JSON and caught by a secret scanner. Fix: redact to a placeholder, keep real keys only in n8n, and rotate the exposed key.
- **Deprecated Messenger tags.** Sending with `POST_PURCHASE_UPDATE` failed. Fix: send `messaging_type: RESPONSE` inside the 24-hour window.
- **Unreliable referral links.** `m.me?ref=` often returned a null ref. Fix: switched to matching on the order code in the message text, which always arrives.
- **Version-sensitive AI nodes.** n8n's LangChain nodes shift between releases, so RAG workflows are best built node-by-node and verified, not blindly imported.

12. Glossary

Term	Meaning
Webhook	A URL that waits to receive an HTTP request and starts a workflow.
RLS (Row Level Security)	Database rules that restrict which rows a user can read/write.
RAG	Retrieval-Augmented Generation — retrieve facts, then let the LLM answer from them.
Embedding	A numeric vector representing the meaning of text, for similarity search.
pgvector	A Postgres extension that stores and searches embedding vectors.
PSID	Page-Scoped ID — a user's unique id for one Facebook Page; needed to message them.
E.164	International phone format, e.g. +639171234567 .
OAuth	Sign-in standard that lets users log in with Google without sharing a password.
Idempotent	Safe to repeat with the same result (GET is; POST usually isn't).

13. Interview Talking Points & Likely Questions

What this project demonstrates

- **Automation:** designed and debugged real n8n workflows with webhooks, data transforms, external API calls, and error paths.
- **Applied AI:** a working RAG pipeline (vector DB + embeddings + LLM + prompt design) and an LLM drafting feature.
- **Integration thinking:** app and automations are decoupled behind webhooks, so either side can change independently.
- **System design:** multi-tenant data with database-enforced isolation.
- **Judgement:** chose the right channel per constraint, and dropped a flaky approach (referral) for a reliable one (code-in-message).

Questions an interviewer might ask

Question	How to answer
Why n8n instead of writing a backend?	It keeps automation visual and swappable, keeps API keys off the frontend, and is the exact skill the role targets. The app stays a thin client.
How do you keep one shop from seeing another's data?	Row Level Security in Postgres — every query is filtered by <code>shop_id = auth.uid()</code> in the database, not the UI.
How does the chatbot avoid making things up?	RAG: it only answers from retrieved FAQ chunks, with a system prompt that says "if it's not in the context, say you're not sure."
Difference between the Webhook and HTTP Request nodes?	Webhook receives (n8n is the server / trigger); HTTP Request sends (n8n is the client, mid-workflow).
Why SMS and Messenger both?	SMS works cold (no opt-in); Messenger is free and richer but needs opt-in + a 24-hour window. Channel matched to constraint.
What was the hardest bug?	The Messenger PSID linking — referral events were unreliable, so I switched to matching the order code in the message text.
How would you scale the chatbot to many shops?	Filter the vector search by a <code>shop_id</code> in document metadata so each shop only retrieves its own FAQ.

Washify — built as a portfolio project. Not affiliated with any real laundry business.